

# Experiment 16

**Name:** N. PARNITA

**Course Code:** MLA0305

**Regno:** 192525322

**Slot:** B

**Title:** Reinforcement Learning Application Development and Performance Evaluation using TensorFlow/Keras

## Software Required:

- Python
- Google Colab
- TensorFlow
- keras
- Git
- GitHub

## AIM

To develop a **Reinforcement Learning application using TensorFlow/Keras** and evaluate its performance based on the rewards obtained during training.

## THEORY

Reinforcement Learning is a machine learning approach in which an **agent interacts with an environment** by taking actions and receiving rewards. The agent learns a policy that helps it choose better actions over time.

TensorFlow and Keras can be used to build and train neural-network-based RL models. Performance can be evaluated using measures such as **episode reward, average reward, and training progress**.

## PROCEDURE

1. Open Google Colab and create a new Python notebook.
2. Import TensorFlow, Keras, NumPy and other required libraries.
3. Define a suitable Reinforcement Learning environment.

4. Identify the state space and action space of the environment.
5. Create the neural network model using TensorFlow/Keras.
6. Initialize the model parameters and optimizer.
7. Allow the agent to interact with the environment.
8. Train the Reinforcement Learning model using the collected experience.
9. Record the reward obtained during each episode.
10. Evaluate the trained model and visualize its performance using a graph.

## CODE:

```
# EXPERIMENT 16
# Reinforcement Learning Application using TensorFlow/Keras
```

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
```

```
# -----
# 1. Create RL Environment
# -----
```

```
num_states = 5
num_actions = 2
```

```
# Reward table
reward_table = np.array([
    [1, 0],
    [0, 2],
    [1, 1],
    [0, 3],
    [2, 1]
], dtype=np.float32)
```

```
# -----
# 2. Create Keras Model
# -----
```

```
model = keras.Sequential([
    layers.Input(shape=(num_states,)),
```

```

        layers.Dense(16, activation="relu"),
        layers.Dense(num_actions, activation="softmax")
    ])

optimizer = keras.optimizers.Adam(learning_rate=0.01)

# -----
# 3. Training
# -----

episodes = 20
episode_rewards = []

for episode in range(episodes):

    # Select random state
    state = np.random.randint(num_states)

    # One-hot state
    state_input = np.zeros((1, num_states), dtype=np.float32)
    state_input[0, state] = 1.0

    with tf.GradientTape() as tape:

        probabilities = model(
            state_input,
            training=True
        )

        # Select action
        action = tf.random.categorical(
            tf.math.log(probabilities),
            1
        )

        action = int(action[0, 0])

        # Get reward
        reward = reward_table[state, action]

        # Policy loss
        loss = -tf.math.log(

```

```

        probabilities[0, action] + 1e-8
    ) * reward

# Update model
gradients = tape.gradient(
    loss,
    model.trainable_variables
)

optimizer.apply_gradients(
    zip(gradients, model.trainable_variables)
)

episode_rewards.append(float(reward))

print(
    "Episode",
    episode + 1,
    "| Reward:",
    int(reward)
)

# -----
# 4. Performance Graph
# -----

plt.figure(figsize=(9, 5))

plt.plot(
    range(1, episodes + 1),
    episode_rewards,
    marker="o",
    label="TensorFlow/Keras RL"
)

plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Reinforcement Learning Performance")

plt.legend()
plt.grid(True)

plt.show()

```

```

# -----
# 5. Final Result
# -----

print("\n=====")
print("EXPERIMENT 16 COMPLETED")
print("=====")

print(
    "Best Reward:",
    max(episode_rewards)
)

print(
    "Average Reward:",
    round(np.mean(episode_rewards), 2)
)

```

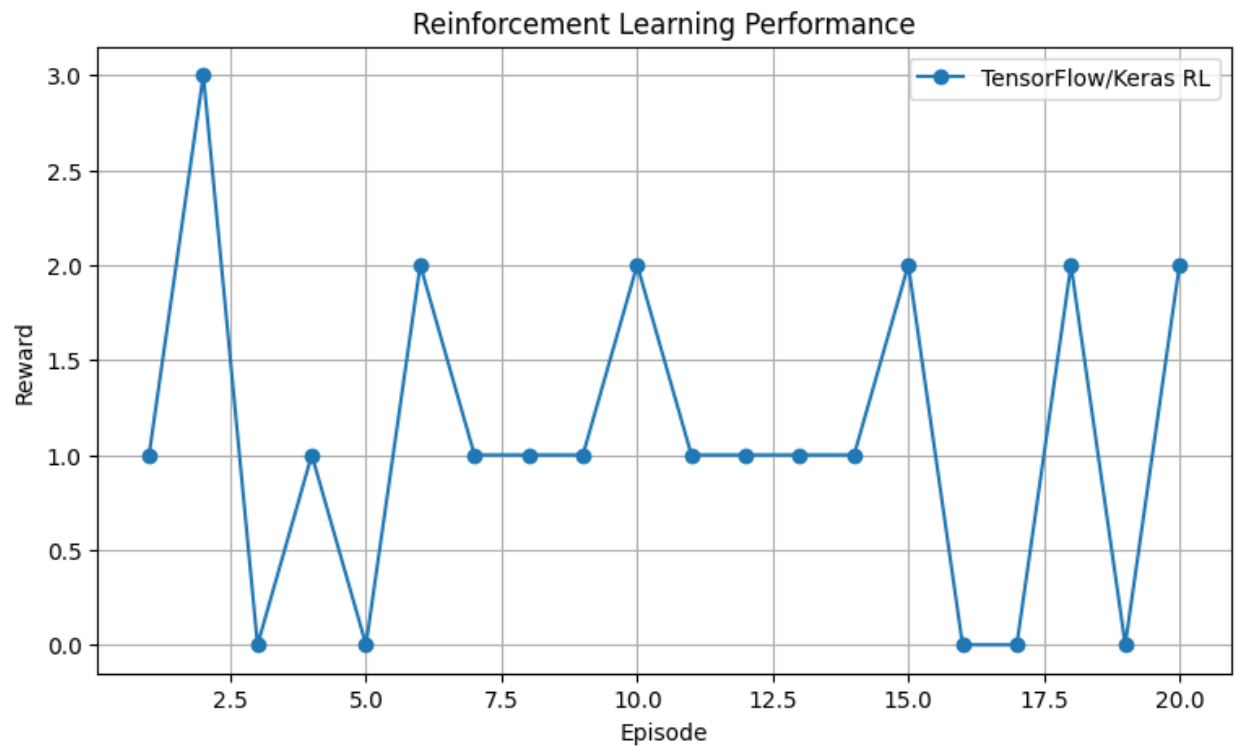
## OUTPUT:

```

Episode 1 | Reward: 1
Episode 2 | Reward: 3
Episode 3 | Reward: 0
Episode 4 | Reward: 1
Episode 5 | Reward: 0
Episode 6 | Reward: 2
Episode 7 | Reward: 1
Episode 8 | Reward: 1
Episode 9 | Reward: 1
Episode 10 | Reward: 2
Episode 11 | Reward: 1
Episode 12 | Reward: 1
Episode 13 | Reward: 1
Episode 14 | Reward: 1
Episode 15 | Reward: 2
Episode 16 | Reward: 0
Episode 17 | Reward: 0
Episode 18 | Reward: 2
Episode 19 | Reward: 0

```

Episode 20 | Reward: 2



=====  
EXPERIMENT 16 COMPLETED  
=====  
Best Reward: 3.0  
Average Reward: 1.1

## RESULT:

The Reinforcement Learning application was successfully developed using **TensorFlow/Keras**, and its performance was evaluated using the rewards obtained during training.